Theory of Computation

Lecture 15

Properties of Programs

Syntactic	Semantic
Number of lines of code.	Reachability of LOC.
Use of while loops.	Termination.
Well balanced parantheses.	Correctness.

Properties of Turing Machines

Syntactic	Semantic
Number of lines of states.	Reachability of states.
Size of tape alphabet.	Halting.
Number of transitions.	Language Properties.

Rice's Theorem

Let P be a non-trivial property of recursively enumerable languages.

$$P : \mathrm{RE}(\Sigma) \rightarrow \{0,1\}$$

Henry Gordon Rice

Rice's Theorem

Let P be a non-trivial property of recursively enumerable languages.

That is, there exists a language L_1 for which it holds and a language L_2 for which it does not hold.

Henry Gordon Rice

Rice's Theorem

Let P be a non-trivial property of recursively enumerable languages.

$$P : \mathrm{RE}(\Sigma) \to \{0,1\}$$

$\exists L_1, L_2 \in \mathrm{RE}(\Sigma)$ such that

$$P(L_1) \neq P(L_2).$$

Henry Gordon Rice

Rice's Theorem

Let P be a non-trivial property of recursively enumerable languages.

Then $L_P = \{\langle M \rangle : P(\mathscr{L}(M)) \text{ holds}\}$ is not recursive.

Henry Gordon Rice

Rice's Theorem

$$\mathrm{HALT}_{\mathrm{TM}} \leq_T \{\langle M \rangle : P(M) \text{ holds}\}$$

Suppose P holds for M_1.

Consider $\langle M, w \rangle$ be an arbitrary instance of the halting problem.

Construct $A_{M,w}$ such that on input x,

$A_{M,w}$ simulates M on w.

If M halts on w, check if $x \in \mathscr{L}(M_1)$.

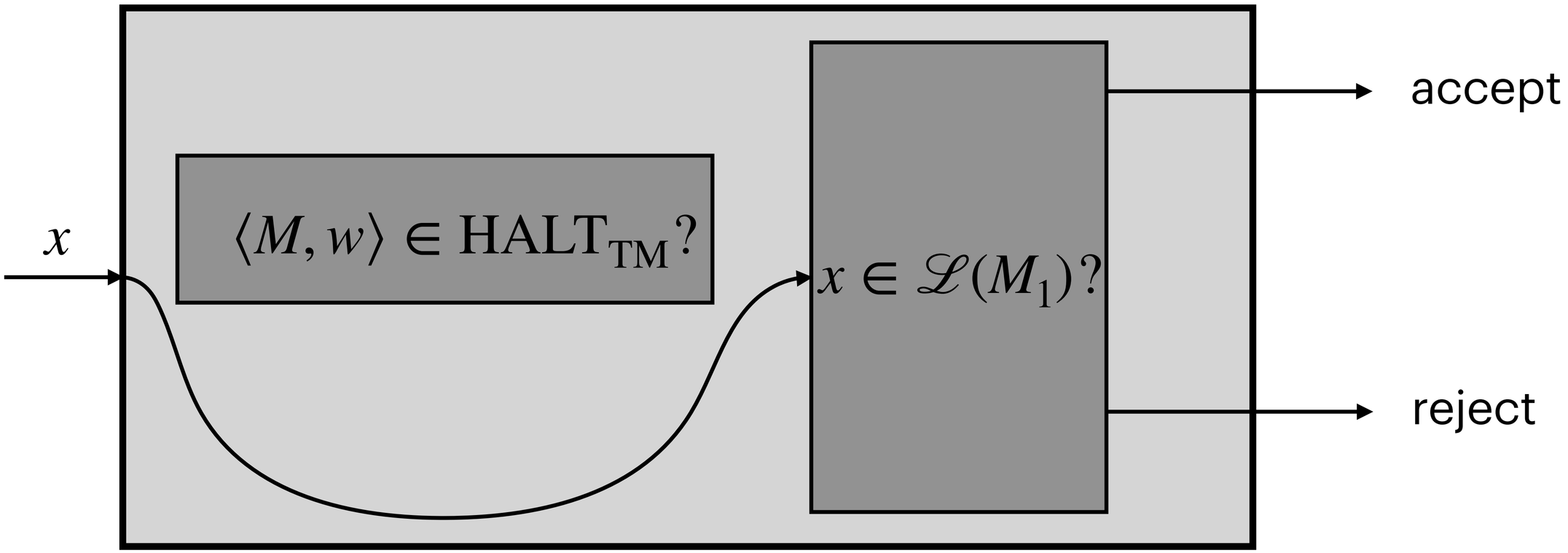
Rice's Theorem

$$\mathrm{HALT}_{\mathrm{TM}} \leq_T \{\langle M \rangle : P(M) \text{ holds}\}$$

Suppose P holds for M_1.

$$\langle M, w \rangle \in \mathrm{HALT}_{\mathrm{TM}} \iff \langle M' \rangle \in \{\langle M \rangle : P(M) \text{ holds}\}$$

Rice's Theorem



$\mathscr{L}(M_1)$ has property P

Rice's Theorem

$$\mathrm{HALT}_{\mathrm{TM}} \leq_T \{\langle M \rangle : P(M) \text{ holds}\}$$

Suppose P holds for M_1.

$$\langle M, w \rangle \in \mathrm{HALT}_{\mathrm{TM}} \iff \langle M' \rangle \in \{\langle M \rangle : P(M) \text{ holds}\}$$

Why does this work?

Rice's Theorem

```
function foo()

   Simulate M on w.


   print "Hello, World!"
end function
```

Applications

The following are undecidable:

$$\{\langle M\rangle : \mathscr{L}(M) \text{ is empty}\}$$

$$\{\langle M\rangle : \mathscr{L}(M) \text{ is finite}\}$$

$$\{\langle M\rangle : \mathscr{L}(M) \text{ is regular}\}$$

$$\{\langle M\rangle : \mathscr{L}(M) \text{ is context-free}\}$$

$$\{\langle M\rangle : \mathscr{L}(M) \text{ is recursive}\}$$

Applications

The following are decidable:

$$\{\langle M \rangle : M \text{ has 5 states}\}$$

$$\{\langle M \rangle : \mathscr{L}(M) \text{ is recursively enumerable}\}$$

$$\{\langle M \rangle : \mathscr{L}(M) \text{ is uncountable}\}$$

Model-Agnostic Time Complexity

Folk-lore: If a deterministic model of computation can solve a problem in $O(T(n))$, then a deterministic Turing Machine can solve it in $O(\mathrm{poly}(T(n)))$.

Hence we study the class of deterministic polynomial time instead of linear time, quadratic time, etc.

Model-Agnostic Time Complexity

Let $\mathbf{P}$ denote the collection of all recursive languages for which membership can be decided in *polynomial time* by a deterministic Turing Machine.

Model-Agnostic Time Complexity

Let $\mathbf{P}$ denote the collection of all recursive languages for which membership can be decided in *polynomial time* by a deterministic Turing Machine.

Prove that $\mathbf{P}$ is closed under the following:

Boolean Combination, Concatenation, Kleene star, Homomorphisms

Model-Agnostic Time Complexity

Let $\mathbf{NP}$ denote the collection of all recursive languages for which membership can be decided in *polynomial time* by a non-deterministic Turing Machine.

Prove that $\mathbf{NP}$ is closed under the following:

Boolean Combination, Concatenation, Kleene star, Homomorphisms

Model-Agnostic Time Complexity

Let $\mathbf{NP}$ denote the collection of all recursive languages for which membership can be decided in *polynomial time* by a non-deterministic Turing Machine.

Prove that $\mathbf{NP}$ is closed under the following:

Boolean Combination, Concatenation, Kleene star, Homomorphisms

Additionally $\mathbf{P} \subseteq \mathbf{NP}$.

Turing Reductions

Suppose we have $L_1 \subseteq \Sigma^*$ and $L_2 \subseteq \Gamma^*$,
and a function $f : \Sigma^* \to \Gamma^*$ such that:

1. f is computable
2. $x \in L_1 \iff f(x) \in L_2$

Then we say $L_1 \leq_T L_2$.

If L_2 is recursive then so is L_1.

Karp Reductions

Suppose we have $L_1 \subseteq \Sigma^*$ and $L_2 \subseteq \Gamma^*$,
and a function $f : \Sigma^* \to \Gamma^*$ such that:

1. f can be computed in polynomial time.
2. $x \in L_1 \iff f(x) \in L_2$

Then we say $L_1 \leq_P L_2$.

Clearly: $L_2 \in \mathbf{P} \implies L_1 \in \mathbf{P}$

Karp Reductions

Suppose we have $L_1 \subseteq \Sigma^*$ and $L_2 \subseteq \Gamma^*$,
and a function $f : \Sigma^* \to \Gamma^*$ such that:

1. f can be computed in polynomial time.
2. $x \in L_1 \iff f(x) \in L_2$

Then we say $L_1 \leq_P L_2$.

Clearly: $L_2 \in \mathbf{NP} \implies L_1 \in \mathbf{NP}$

Karp Reductions

$\leq_P$ forms a quasi-order.

Therefore $\mathbf{P}$ and $\mathbf{NP}$ are both downward closed under $\leq_P$.

Verification Definition

The following are equivalent:

L is decided by a polynomial-time nondeterministic Turing machine

L has a polynomial-time verifier with polynomial-size certificate

Verification Definition

Let $L \in \mathbf{NP}$.

Then there is a NTM M such that:

1. $\mathscr{L}(M_1) = L$
2. For all $x \in L$, M runs in $p(|x|)$ time.

The sequence of choices can be written as a string y.

Verification Definition

Let $L \in \mathbf{NP}$.

The sequence of choices can be written as a string y.

y is of polynomial size.

Let $V(x, y)$ be a deterministic verifier.

Verification Definition

Let $L \in \mathbf{NP}$.

The sequence of choices can be written as a string y.

y is of polynomial size.

Let $V(x, y)$ be a deterministic verifier.

At each step, make the move of M, according to y.

If this simulation of M, is accepted then accept. Else reject.

Verification Definition

Let L have a verifier V.

For each $x \in L$, there is a y is of polynomial size such that $V(x, y)$ is accepted.

Construct machine M that non-deterministically guesses y and runs $V(x, y)$.

If verifier accepts then accept. Else reject.

Verification Definition

$$L \in \mathbf{NP} \iff \exists V \left(\forall x \exists y \text{ such that } |y| \leq p(|x|) \text{ and } V(x, y) \text{ is accepted} \right)$$

The following are in $\mathbf{NP}$:

1. Every problem in $\mathbf{P}$.
2. Hamiltonian Cycle
3. Satifiability of Propositional Logic Formula
4. Solvability of a Sudoku puzzle.

Class-Hard Problems

Let $\mathscr{C} \subseteq \Sigma^*$ be a class of problems (like $\mathbf{P}$ or $\mathbf{NP}$).

A language $H \subseteq \Sigma^*$ is hard for class $\mathscr{C}$ with respect to reductions $\leq_R$

$$\forall L \in \mathscr{C}, L \leq_R H.$$

Class-Hard Problems

Let $\mathscr{C} \subseteq \Sigma^*$ be a class of problems (like $\mathbf{P}$ or $\mathbf{NP}$).

A language $H \subseteq \Sigma^*$ is hard for class $\mathscr{C}$ with respect to reductions $\leq_R$

$$\forall L \in \mathscr{C}, L \leq_R H.$$

$$\forall L, H \in \mathbf{P}, L \leq_P H.$$

NP-Hard Problems

A language $H \subseteq \Sigma^*$ is NP-hard if

$$\forall L \in \mathbf{NP}, L \leq_P H.$$

A language $H \subseteq \Sigma^*$ is NP-hard if

$$\exists L \in \mathbf{NP}\text{-hard}, L \leq_P H.$$

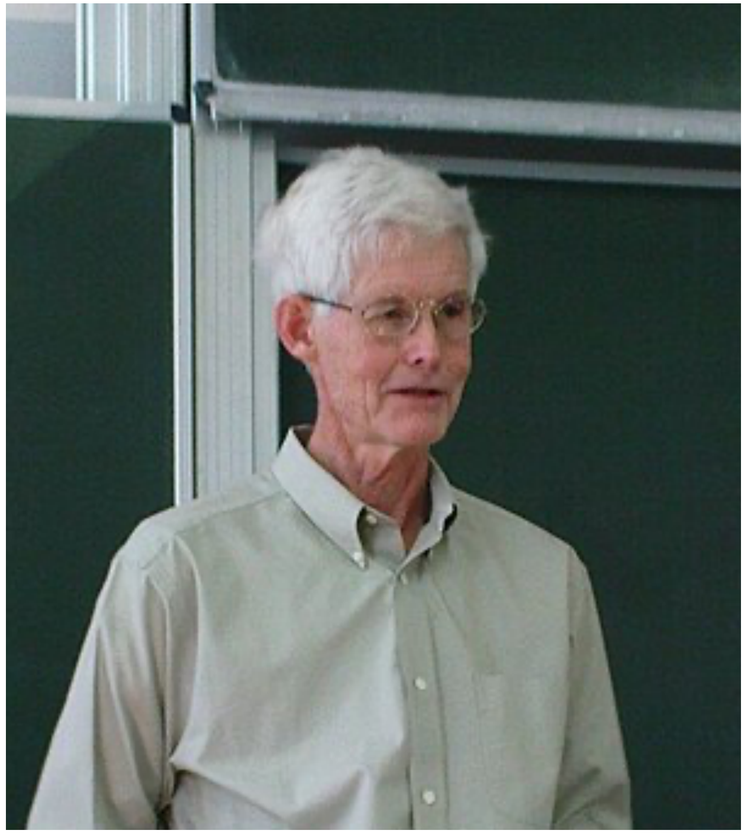
NP-Complete Problems

$$\mathbf{NP}\text{-complete} = \mathbf{NP} \cap \mathbf{NP}\text{-hard}$$

A language $H \subseteq \Sigma^*$ is NP-complete if

$H \in \mathbf{NP}$ and $H \in \mathbf{NP}$-hard.

The First NP-complete Problem



Stephen Cook

Leonid Levin

Stephen Cook. *The Complexity of Theorem Proving Procedures* (1971).
Leonid Levin. *Problems of Information Transmission* (1973).

The First NP-complete Problem

$$\mathrm{NTM}_{\mathrm{ACCEPT}} = \{\langle M, w, 1^t \rangle : M \text{ accepts } w \text{ in } t \text{ steps}\}$$

$$\text{P-NTM} = \bigcup_{p \in \mathrm{poly}} \{\langle M, w, 1^{p(|w|)} \rangle : M \text{ accepts } w \text{ in } p(|w|) \text{ steps}\}$$

Cook-Levin Theorem

$$\text{P-NTM} \leq_P \text{SAT}$$

We convert the computation of M into a Boolean formula.

The formula will be satisfiable iff the machine has an accepting computation.

Cook-Levin Theorem

$$\text{P-NTM} \leq_P \text{SAT}$$

Given $\langle M, w, 1^t \rangle \in \text{P-NTM}$, consider the variable $X_{t,i,s}$ to mean that at time t, cell i contains symbol s.

How many variables?

$$\phi = \phi_{\text{initial}} \wedge \phi_{\text{move}} \wedge \phi_{\text{cell}} \wedge \phi_{\text{accept}}$$

Cook-Levin Theorem

$$\text{P-NTM} \leq_P \text{SAT}$$

Given $\langle M, w, 1^t \rangle \in \text{P-NTM}$, consider the variable $X_{t,i,s}$ to mean that at time t, cell i contains symbol s.

How many variables?

$$\phi \in SAT \iff \langle M, w, 1^t \rangle \in \text{P-NTM}$$

Cook-Levin Theorem

$$\text{P-NTM} \leq_P \text{SAT}$$

Therefore, $\text{P-NTM} \leq_P \text{SAT}$.

Clearly, $SAT \in \mathbf{NP}$.

Cook-Levin Theorem

$$\text{P-NTM} \leq_P \text{SAT}$$

Therefore, $\text{P-NTM} \leq_P \text{SAT}$.

Hence $SAT \in \mathbf{NP}$-complete.

The Boolean Satisfiability Problem

$$(A \vee B) \wedge (A \leftarrow (C \rightarrow D)) \wedge (\neg(D \oplus A)) \wedge (B \leftrightarrow D)$$

$$(A \vee B) \wedge (A \vee \neg(\neg C \vee D)) \wedge (\neg((D \vee A) \wedge \neg(D \wedge A))) \wedge ((B \wedge D) \vee (\neg B \wedge \neg D))$$

Conjunctive Normal Form

$$(A \vee B) \wedge (A \leftarrow (C \rightarrow D)) \wedge (\neg(D \oplus A)) \wedge (B \leftrightarrow D)$$

$$(A \vee B) \wedge (A \vee \neg(\neg C \vee D)) \wedge (\neg((D \vee A) \wedge \neg(D \wedge A))) \wedge ((B \wedge D) \vee (\neg B \wedge \neg D))$$

$$p = c \mid c \wedge p$$

$$c = l \mid l \vee c$$

$$l = A \mid \neg A$$

Normalisation Theorem: For every propositional formula, there exists an *equivalent* formula in conjunctive normal form.

Conjunctive Normal Form

$$(A \vee B) \wedge (A \leftarrow (C \rightarrow D)) \wedge (\neg(D \oplus A)) \wedge (B \leftrightarrow D)$$

$$(A \vee B) \wedge (A \vee \neg(\neg C \vee D)) \wedge (\neg((D \vee A) \wedge \neg(D \wedge A))) \wedge ((B \wedge D) \vee (\neg B \wedge \neg D))$$

$$A \vee B$$

$$A \vee \neg(\neg C \vee D) \implies A \vee (C \wedge \neg D) \implies (A \vee C) \wedge (A \vee \neg D)$$

$$\neg((D \vee A) \wedge \neg(D \wedge A)) \implies (\neg D \wedge \neg A) \vee (D \wedge A) \implies$$

$$((\neg D \wedge \neg A) \vee D) \wedge ((\neg D \wedge \neg A) \vee A) \implies$$

$$(\neg D \vee D) \wedge (\neg A \vee D) \wedge (\neg D \vee A) \wedge (\neg A \vee A)$$

$$\ldots$$

Conjunctive Normal Form

$$(A \vee B) \wedge (A \leftarrow (C \rightarrow D)) \wedge (\neg(D \oplus A)) \wedge (B \leftrightarrow D)$$

$$(A \vee B) \wedge (A \vee \neg(\neg C \vee D)) \wedge (\neg((D \vee A) \wedge \neg(D \wedge A))) \wedge ((B \wedge D) \vee (\neg B \wedge \neg D))$$

$$(A \vee B) \wedge (A \vee C) \wedge (A \vee \neg D), (\neg A \vee D) \wedge (\neg D \vee A) \wedge (B \vee \neg D) \wedge (D \vee \neg B)$$

$$CNF\text{-}SAT \leq_P SAT$$

$$SAT \leq_P CNF\text{-}SAT\,?$$

CNF-SAT

$$(A_1 \wedge B_1) \vee (A_2 \wedge B_2) \vee \ldots \vee (A_n \wedge B_n)$$

$$(A_1 \vee A_2 \vee \ldots \vee A_n) \wedge (B_1 \vee A_2 \vee \ldots \vee A_n) \wedge \ldots \wedge (B_1 \vee B_2 \vee \ldots \vee B_n)$$

Tseitin Theorem: For every formula, there exists an *equisatifiable* formula in 3-CNF. The transformation can be done in polynomial time.

CNF-SAT

$$(A_1 \wedge B_1) \vee (A_2 \wedge B_2) \vee \ldots \vee (A_n \wedge B_n)$$

$$C_1 \qquad C_2 \qquad C_n$$

$$\begin{array}{ccc} \neg C_1 \vee A_1 & \neg C_2 \vee A_2 & \neg C_n \vee A_n \\ \neg C_1 \vee B_1 & \neg C_2 \vee B_2 & \neg C_n \vee B_n \end{array}$$

$$(C_1 \vee C_2 \vee \ldots \vee C_n) \wedge (\neg C_1 \vee A_1), (\neg C_1 \vee B_1) \wedge \ldots \wedge (\neg C_n \vee A_n) \wedge (\neg C_n \vee B_n)$$

CNF-SAT

$$(A_1 \wedge B_1) \vee (A_2 \wedge B_2) \vee \ldots \vee (A_n \wedge B_n) \qquad 2n$$

$$(A_1 \vee A_2 \vee \ldots \vee A_n) \wedge (B_1 \vee A_2 \vee \ldots \vee A_n) \wedge \ldots \wedge (B_1 \vee B_2 \vee \ldots \vee B_n) \qquad O(2^n)$$

$$(C_1 \vee C_2 \vee \ldots \vee C_n) \wedge (\neg C_1 \vee A_1), (\neg C_1 \vee B_1) \wedge \ldots \wedge (\neg C_n \vee A_n) \wedge (\neg C_n \vee B_n) \qquad O(n)$$

There is no free lunch!

CNF-SAT

$$(A_1 \wedge B_1) \vee (A_2 \wedge B_2) \vee \ldots \vee (A_n \wedge B_n) \qquad 2n$$

$$(A_1 \vee A_2 \vee \ldots \vee A_n) \wedge (B_1 \vee A_2 \vee \ldots \vee A_n) \wedge \ldots \wedge (B_1 \vee B_2 \vee \ldots \vee B_n) \qquad O(2^n)$$

$$(C_1 \vee C_2 \vee \ldots \vee C_n) \wedge (\neg C_1 \vee A_1), (\neg C_1 \vee B_1) \wedge \ldots \wedge (\neg C_n \vee A_n) \wedge (\neg C_n \vee B_n) \qquad O(n)$$

$$CNF\text{-}SAT \leq_P SAT$$

$$SAT \leq_P CNF\text{-}SAT\,?$$

3-CNF-SAT (3-SAT)

Tseitin Theorem: For every formula, there exists an *equisatifiable* formula in 3-CNF. The transformation can be done in **polynomial time**.

$$A \vee B \vee C \vee D$$

3-CNF-SAT (3-SAT)

Tseitin Theorem: For every formula, there exists an *equisatifiable* formula in 3-CNF. The transformation can be done in **polynomial time**.

$$E \vee F$$

$$(E \rightarrow A \vee B) \wedge (F \rightarrow C \vee D)$$

3-CNF-SAT (3-SAT)

Tseitin Theorem: For every formula, there exists an *equisatifiable* formula in 3-CNF. The transformation can be done in **polynomial time**.

$$E \vee F$$

$$(\neg E \vee A \vee B) \wedge (\neg F \vee C \vee D)$$

$$3\text{-}SAT \in \mathbf{NP}\text{-complete}$$

Boolean Satisfiability

$$(A \vee B) \wedge (A \leftarrow (C \rightarrow D)) \wedge (\neg(D \oplus A)) \wedge (B \leftrightarrow D)$$

$$(A \vee B) \wedge (A \vee C) \wedge (A \vee \neg D), (\neg A \vee D) \wedge (\neg D \vee A) \wedge (B \vee \neg D) \wedge (D \vee \neg B)$$

Can we check satisfiability in linear time?

Boolean Satisfiability

$$(A \vee B) \wedge (A \leftarrow (C \rightarrow D)) \wedge (\neg(D \oplus A)) \wedge (B \leftrightarrow D)$$

$$(A \vee B) \wedge (A \vee C) \wedge (A \vee \neg D), (\neg A \vee D) \wedge (\neg D \vee A) \wedge (B \vee \neg D) \wedge (D \vee \neg B)$$

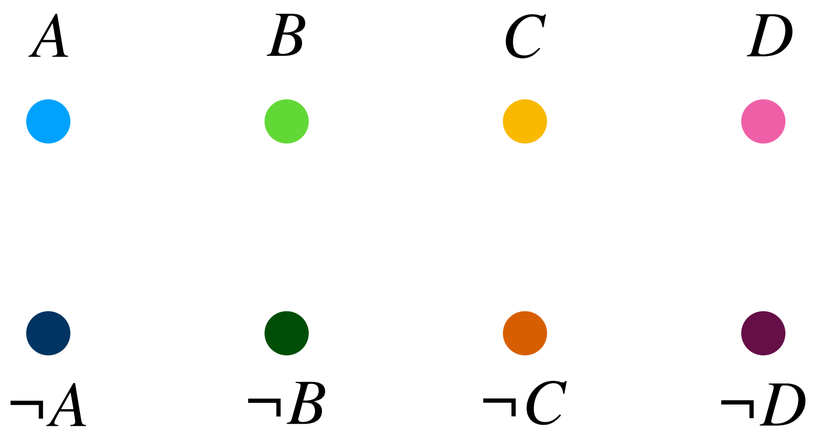
$$\neg A \rightarrow B$$
$$\neg A \rightarrow C$$
$$D \rightarrow A$$
$$A \rightarrow D$$
$$B \rightarrow D$$
$$D \rightarrow B$$

Boolean Satisfiability

$$(A \vee B) \wedge (A \leftarrow (C \rightarrow D)) \wedge (\neg(D \oplus A)) \wedge (B \leftrightarrow D)$$

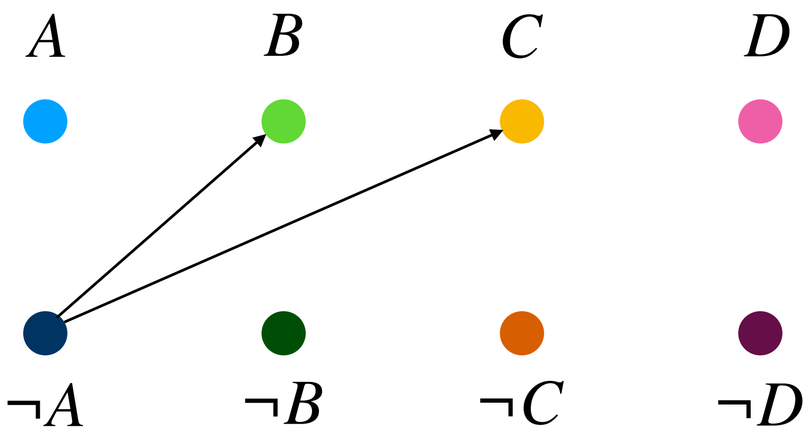
$$(A \vee B) \wedge (A \vee C) \wedge (A \vee \neg D), (\neg A \vee D) \wedge (\neg D \vee A) \wedge (B \vee \neg D) \wedge (D \vee \neg B)$$

$\neg A \rightarrow B$	$\neg B \rightarrow A$
$\neg A \rightarrow C$	$\neg C \rightarrow A$
$D \rightarrow A$	$\neg A \rightarrow \neg D$
$A \rightarrow D$	$\neg D \rightarrow \neg A$
$B \rightarrow D$	$\neg D \rightarrow \neg B$
$D \rightarrow B$	$\neg B \rightarrow \neg D$



$$\neg A \to B$$
$$\neg A \to C$$
$$D \to A$$
$$A \to D$$
$$B \to D$$
$$D \to B$$

$$\neg B \to A$$
$$\neg C \to A$$
$$\neg A \to \neg D$$
$$\neg D \to \neg A$$
$$\neg D \to \neg B$$
$$\neg B \to \neg D$$



$\neg A \to B$

$\neg A \to C$

$D \to A$

$A \to D$

$B \to D$

$D \to B$

$\neg B \to A$

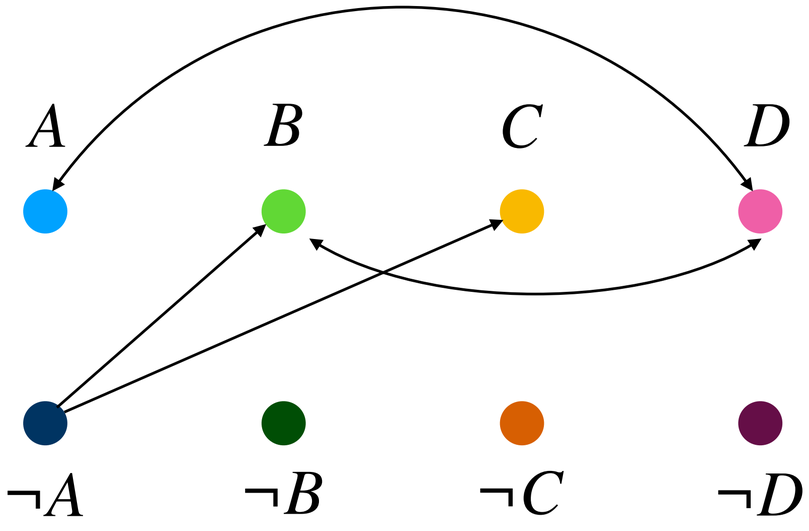
$\neg C \to A$

$\neg A \to \neg D$

$\neg D \to \neg A$

$\neg D \to \neg B$

$\neg B \to \neg D$



$\neg A \to B$

$\neg A \to C$

$D \to A$

$A \to D$

$B \to D$

$D \to B$

$\neg B \to A$

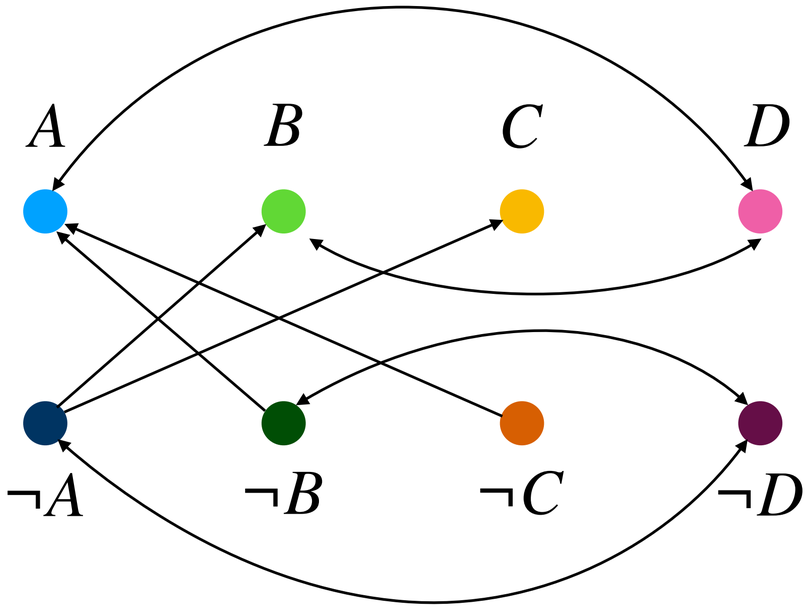
$\neg C \to A$

$\neg A \to \neg D$

$\neg D \to \neg A$

$\neg D \to \neg B$

$\neg B \to \neg D$



$\neg A \to B$

$\neg A \to C$

$D \to A$

$A \to D$

$B \to D$

$D \to B$

$\neg B \to A$

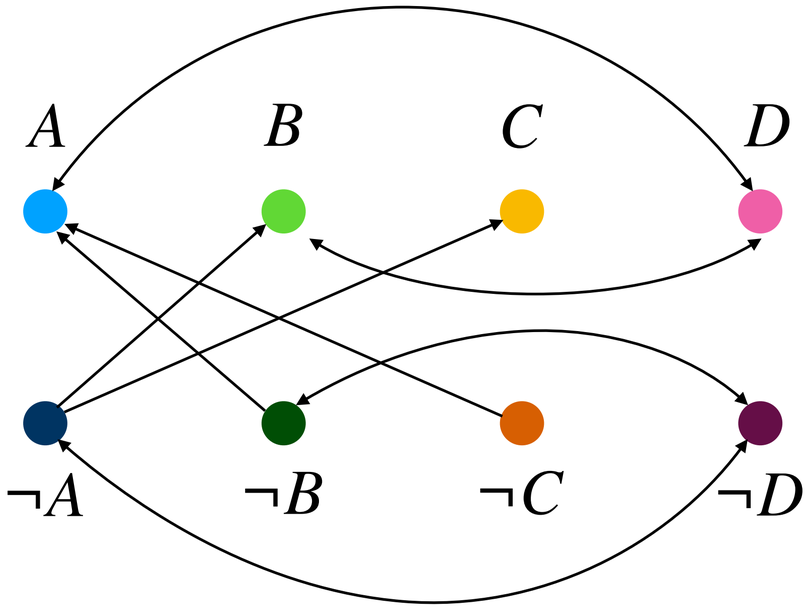
$\neg C \to A$

$\neg A \to \neg D$

$\neg D \to \neg A$

$\neg D \to \neg B$

$\neg B \to \neg D$



Aspvall, Plass, Tarjan (1979): For any variable X, the vertices for X and $\neg X$ exist in a strongly connected component of the implication graph if and only if the set is not satisfiable.

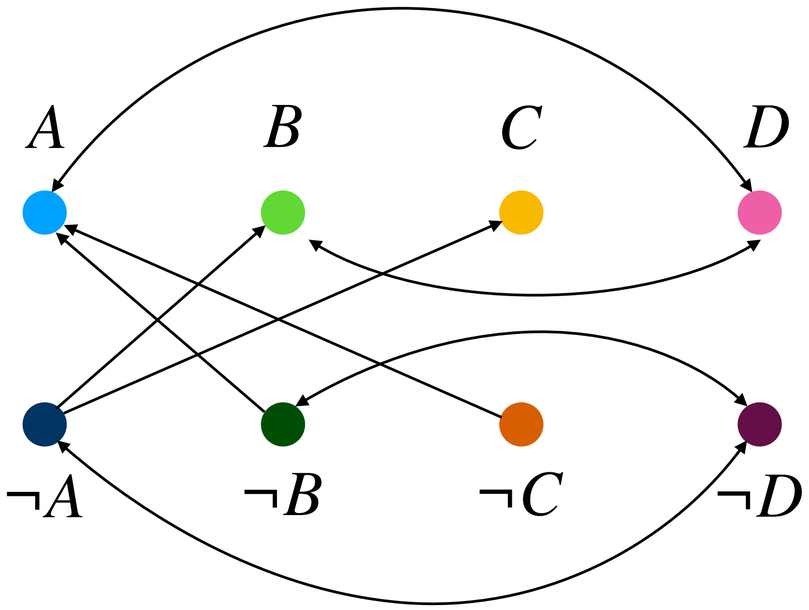
Boolean Satisfiability

Cook-Levin Theorem:

$SAT \in \mathbf{NP}$-complete

Tseitin Theorem:

$3\text{-}SAT \in \mathbf{NP}$-complete



$2\text{-}SAT \in \mathbf{P}$